



RAG-BASED HR ASSISTANT

Professional AI & Data Science Project Report

Retrieval-Augmented Generation • Semantic Search • FAISS • Streamlit

AUTHOR
Reem Fayyaz

2026 • Project Documentation

An intelligent HR knowledge assistant that retrieves relevant organizational policy information and produces grounded answers through a modern RAG architecture.

1. Executive Summary

The RAG-Based HR Assistant is a professional Streamlit application designed to help employees find answers from HR policies quickly and consistently. Rather than relying only on a general-purpose language model, the system first searches a controlled HR knowledge base and retrieves the most relevant passages.

The solution combines SentenceTransformers for semantic embeddings, FAISS for vector similarity search, PyPDF for document extraction, and Streamlit for the interactive web interface. It supports local retrieval and optional OpenAI or Google Gemini answer generation.

Architecture	Retrieval	Interface	Input
RAG	FAISS + Embeddings	Streamlit	PDF + TXT

2. Project Objectives

- Create a professional HR assistant that answers questions from company policy documents.
- Use semantic retrieval instead of simple keyword matching.
- Ground responses in retrieved HR content to reduce unsupported answers.
- Allow HR teams to upload PDF and TXT documents.
- Provide a polished, portfolio-ready Streamlit interface.
- Support local retrieval and optional generative AI providers.

3. Technology Stack

Technology	Purpose
Python	Core programming language
Streamlit	Interactive web application and chat interface
SentenceTransformers	Semantic text embeddings
FAISS	Fast vector similarity search
PyPDF	PDF text extraction
NumPy / Pandas	Numerical and data processing
OpenAI	Optional generative answer layer
Google Gemini	Optional generative answer layer

4. RAG Architecture

The application follows a retrieval-first architecture. This is especially valuable for HR because answers should be grounded in the organization's available policies rather than generated from unsupported assumptions.

1 Documents	2 Chunking	3 Embeddings	4 FAISS	5 Answer
PDF / TXT	Overlapping chunks	MiniLM	Similarity search	Local / AI

5. End-to-End Workflow

1. Document ingestion

Loads built-in HR policies or uploaded PDF/TXT files.

2. Text extraction

Extracts and normalizes document text.

3. Chunk creation

Divides documents into overlapping chunks.

4. Embedding generation

Converts chunks into semantic vectors using all-MiniLM-L6-v2.

5. Vector indexing

Stores embeddings in a FAISS index.

6. Question embedding

Converts the employee question into the same vector space.

7. Top-k retrieval

Selects the most relevant policy chunks.

8. Answer generation

Returns a grounded local response or uses OpenAI/Gemini.

6. Knowledge Base

The application includes demonstration policies for:

- Leave Policy
- Work From Home Policy
- Attendance Policy
- Payroll Policy
- Resignation Policy

Additional organization-specific policies can be uploaded directly from the Streamlit sidebar.

7. User Interface & Experience

- Professional dashboard: shows HR document count, knowledge chunks, vector-search status, and answer mode.
- Chat interface: provides a natural question-and-answer experience.
- Quick questions: provides one-click examples for common HR topics.
- Source transparency: retrieved policy chunks and similarity scores are available after answers.
- Sidebar controls: configure retrieval count, chunk size, overlap, uploads, reset, and chat clearing.

8. Core Implementation Components

Document Loader

Reads PDF/TXT content and normalizes it for processing.

Chunking Engine

Creates overlapping chunks to improve retrieval quality.

Embedding Model

Uses all-MiniLM-L6-v2 to represent semantic meaning as vectors.

FAISS Index

Performs efficient similarity search over normalized vectors.

Retriever

Embeds a question and returns the highest-scoring policy chunks.

Answer Layer

Provides local extractive answers or optional OpenAI/Gemini generation.

Streamlit UI

Provides dashboard, uploader, chat, settings, and source display.

9. Retrieval Strategy

The retrieval layer uses normalized embeddings and FAISS inner-product search. With normalized vectors, the similarity behaves similarly to cosine similarity. The number of retrieved chunks is configurable, with three as the default.

Semantic retrieval allows the system to recognize related language. For example, a user asking about 'remote work' can retrieve a policy that uses the phrase 'work from home'.

10. Answer Safety & Grounding

HR information can affect employees directly, so the application is designed to avoid presenting unsupported policies as facts. If sufficiently relevant information cannot be found, the assistant recommends contacting HR.

When an external generative model is enabled, its prompt instructs it to use the retrieved HR context and acknowledge when the answer is unavailable. This is a core RAG safety principle.

11. Testing & Validation

11. Testing & Validation

Test Area	Expected Result	Status
Application startup	Streamlit app opens successfully	Ready
Built-in policies	Demo policies are indexed	Ready
PDF upload	PDF text is extracted and chunked	Ready

Test Area	Expected Result	Status
TXT upload	TXT text is indexed	Ready
Semantic query	Relevant chunks are retrieved	Ready
Local RAG	Grounded response is returned	Ready
Source display	Source and similarity score are shown	Ready
LLM fallback	Local retrieval remains available if provider fails	Ready

12. Installation & Deployment

Install dependencies:

```
pip install -r requirements.txt
```

```
streamlit run app.py
```

For OpenAI or Gemini, API keys should be stored in environment variables or a secure deployment secrets manager. Never commit API keys to GitHub.

13. Recommended Project Structure

```
RAG-Based-HR-Assistant/ | ├── app.py |── requirements.txt |── README.md |── HR policy documents |── Leave Policy.pdf |── Attendance Policy.pdf |── Work From Home Policy.pdf |── Employee Handbook.pdf
```

14. Security & Privacy Considerations

- Keep API keys in environment variables or platform secrets.
- Do not publish confidential HR documents in public repositories.
- Use access controls when deploying real employee information.
- Keep official HR policies as the source of truth.
- Treat generated answers as assistance, not final HR approval.
- Review privacy and retention requirements before production deployment.

15. Future Enhancements

- Employee authentication and role-based access control.
- Persistent conversation memory.
- DOCX, Excel, and additional enterprise document support.
- OCR for scanned PDFs.
- Department-specific knowledge bases.
- Cloud vector databases for larger collections.
- HR ticket creation and escalation workflows.
- Email and enterprise collaboration integration.
- Document versioning and policy expiration alerts.
- Analytics for frequently asked HR questions.

16. Conclusion

The RAG-Based HR Assistant demonstrates a practical application of modern AI engineering for HR knowledge management. By combining semantic embeddings, FAISS retrieval, document ingestion, and optional generative AI, the project provides a strong foundation for a reliable and scalable HR information assistant.

Its modular design makes it suitable for academic demonstration, a data-science portfolio, or further development into an enterprise HR knowledge platform. The central strength is its grounding mechanism: responses are connected to retrieved organizational policy content rather than generated in isolation.

PROJECT HIGHLIGHT

Retrieval-Augmented Generation + FAISS + SentenceTransformers + Streamlit

Author: Reem Fayyaz